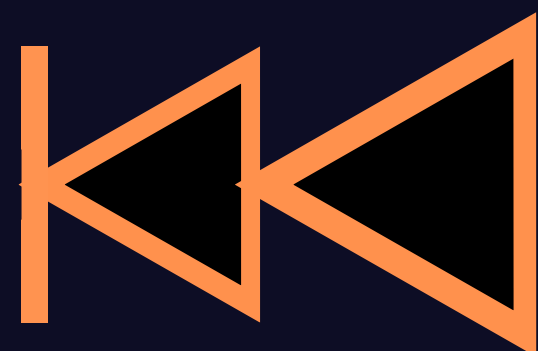




High-level Intro to Reverse Code Engineering

UofTCTF Seminars 2025



Announcements

- Please fill out PSV Interest Form
- If you want to join us for CTFs you can verify with our CTF Bot

Overview

- | | |
|--------------------------------|-----------------------|
| 1. CPU Architecture | 4. Decompilers |
| 2. Compilers &
Interpreters | 5. Debuggers |
| 3. Code Execution | 6. Obfuscation |
| | 7. Symbolic Execution |

CPU Architecture

Instruction Set Architecture (ISA)

A CPU's ISA dictates:

- Registers
- Datatypes
- Allowed Opcodes

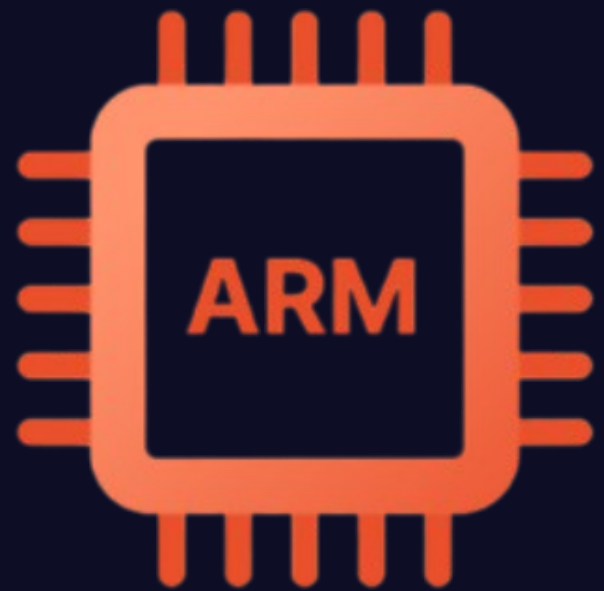
CPU Architecture

Examples of common ISA:



x86:

- Complex (1000+ Instructions)
- Old, everyone uses this



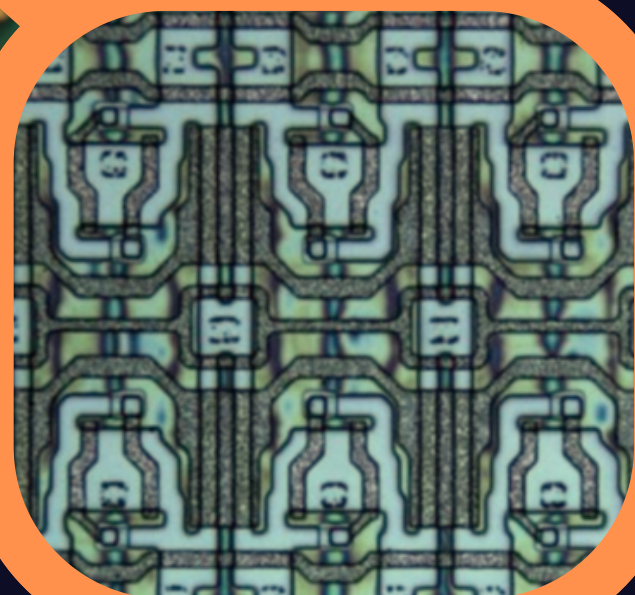
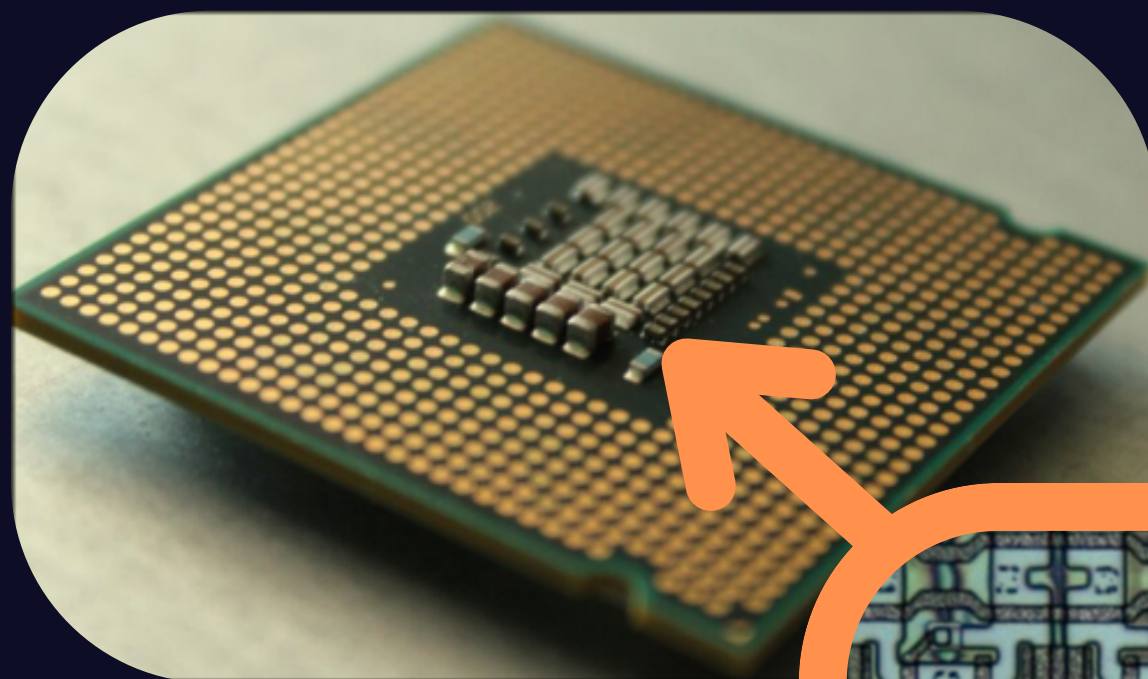
ARM:

- Reduced (100 Instructions)
- New, mostly for apple and qualcomm

CPU Architecture

Registers:

Collection of storage units within the CPU.



Used for quick temporary operations.

CPU Architecture

64-bit	32-bit	16-bit	8 high bits	8 low bits	Description
RAX	EAX	AX	AH	AL	Accumulator
RBX	EBX	BX	BH	BL	Base
RCX	ECX	CX	CH	CL	Counter
RDX	EDX	DX	DH	DL	Data
RSI	ESI	SI	N/A	SIL	Source
RDI	EDI	DI	N/A	DIL	Destination
RSP	ESP	SP	N/A	SPL	Stack Pointer
RBP	EBP	BP	N/A	BPL	Stack Base Pointer

CPU Architecture

Opcode / Machine Code:

A sequence of bytes sent to the CPU for execution of a specific instruction

opcode(in hex)

```
4989d1  
5e  
4889e2  
4883e4f0
```

assembly (human readable)

```
mov     r9, rdx  
pop     rsi {__return_addr}  
mov     rdx, rsp {arg_8}  
and     rsp, 0xffffffffffffff0
```

CPU Architecture

Mneumonics / Assembly:

Intel (x86)

```
mov    r12, rax
mov    r13, rsp
mov    edx, dword ptr [rip + 0x1cc5c]
test   edx, 2
je     0x7ffff7fe0b6d    <0
```

AT&T (x86)

```
movq    %rax, %r12
movq    %rsp, %r13
movl    0x1cc5c(%rip), %edx
testl   $2, %edx
je      0x7ffff7fe0b6d
```


CPU Architecture

How Assembly Executed:

```
► 0x8049000 <_start>      mov     eax, 3
    0x8049005 <_start+5>    mov     ebx, 4
    0x804900a <_start+10>   add     eax, ebx
```

eip : 0x8049000

eax : 0x3

ebx : 0x0

CPU Architecture

How Assembly Executed:

```
0x8049000 <_start>          mov     eax, 3
▶ 0x8049005 <_start+5>      mov     ebx, 4
0x804900a <_start+10>     add     eax, ebx
```

eip : 0x8049005

eax : 0x3

ebx : 0x4

CPU Architecture

How Assembly Executed:

```
0x8049000 <_start>      mov     eax, 3
0x8049005 <_start+5>    mov     ebx, 4
▶ 0x804900a <_start+10> add     eax, ebx
```

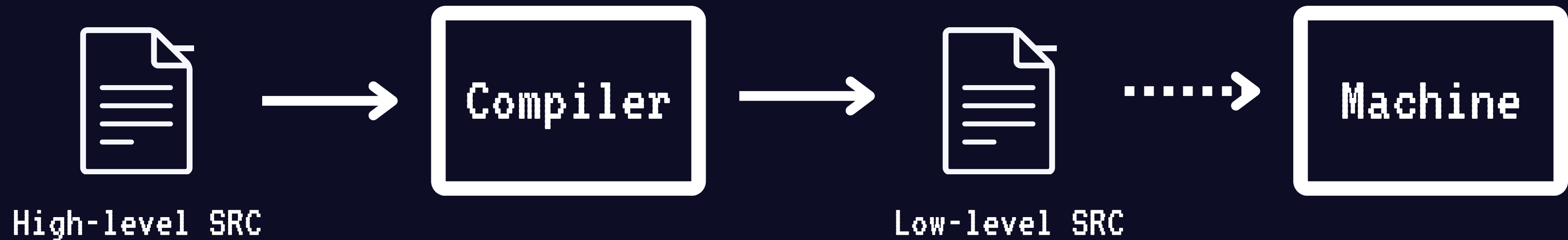
eip : 0x804900a

eax : 0x7

ebx : 0x4

Compilers & Interpreters

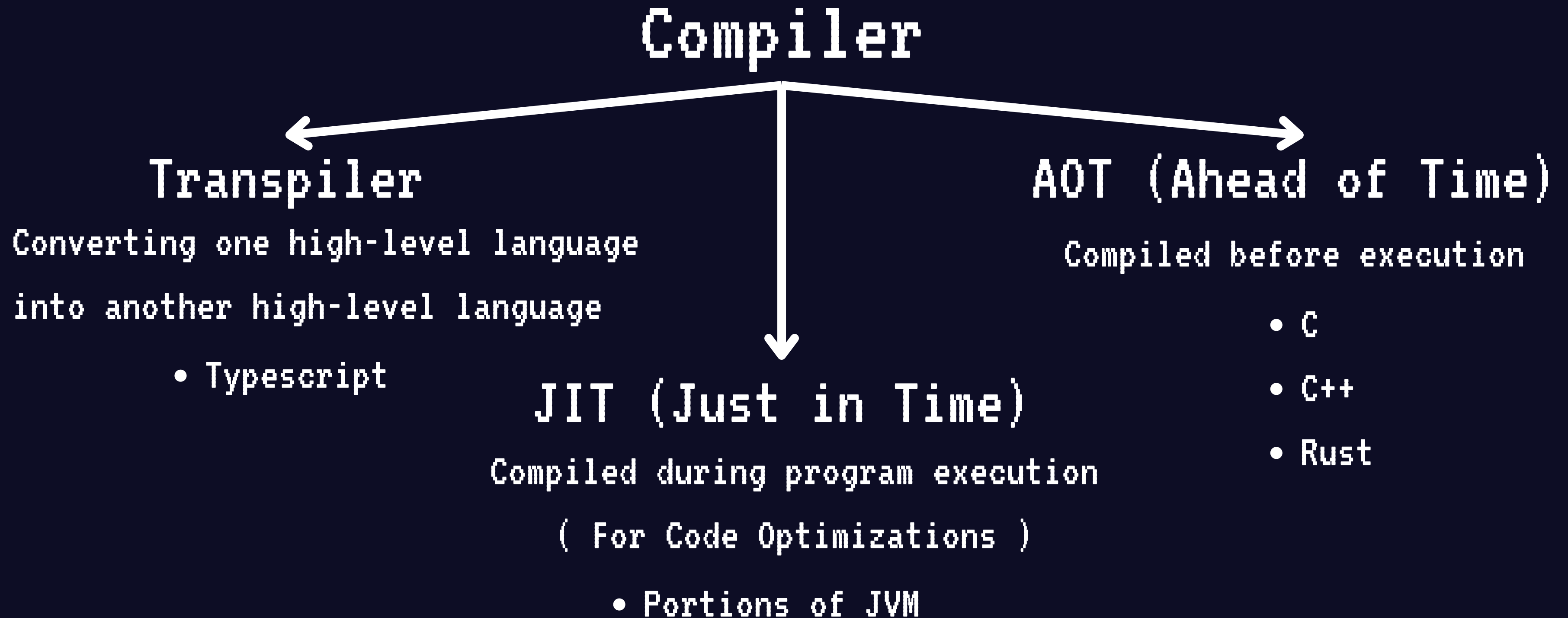
Compilers convert code $\rightarrow$ code



Interpreters execute code



Compilers & Interpreters

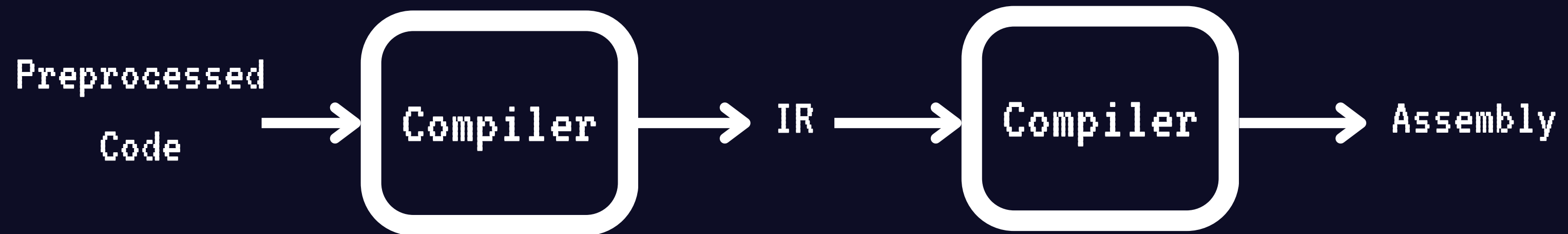


Compilers & Interpreters

Intermediate Representations (IR)

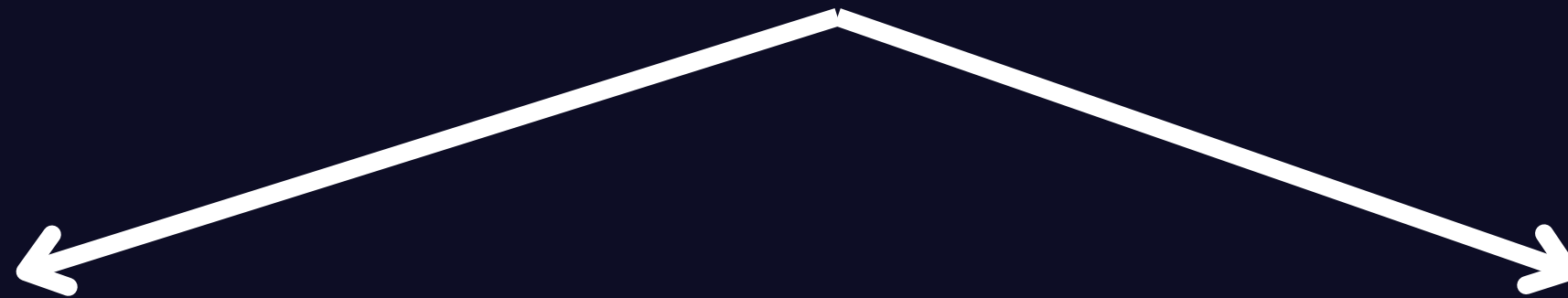
An intermediate language from source code to assembly

- Enforces optimizations (SSA)
- Allows for cross-architecture code



Compilers & Interpreters

Interpreter



Bytecode VM

Convert Sourcecode to bytecode
first, then execute the bytecode

- JVM

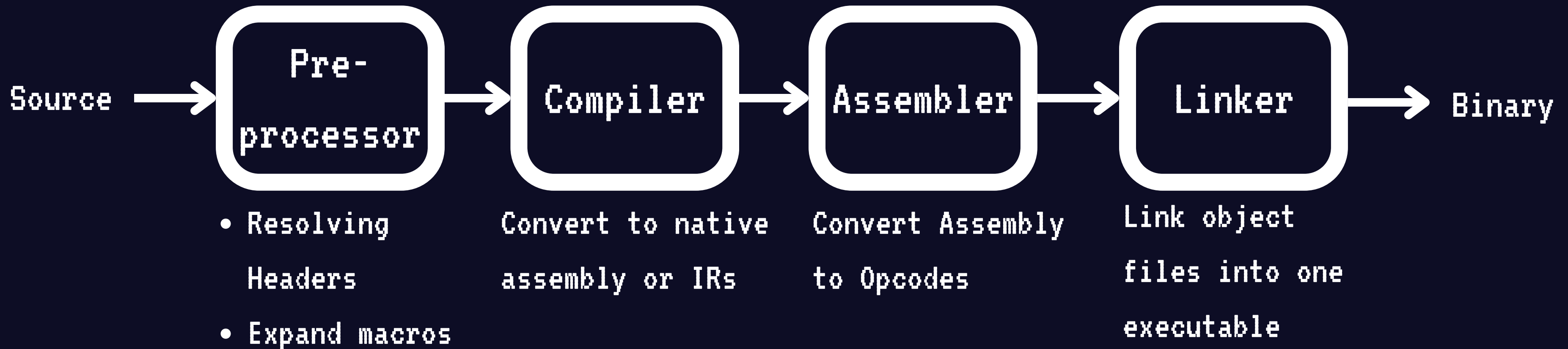
AST

Code is parsed into a AST
and then executed by traversing

- Ruby

Compilers & Interpreters

GCC Toolchain



Compilers & Interpreters

Static Linking

Copying all object files into one file.

main.o

```
110101011000101100111010100011
010110100100010100111111001100
111010110000010101101010011000
110110000101110110011000100110
000000000101011001001001001010
```

stdlib.o

```
110101011000101100111010100011
010110100100010100111111001100
111010110000010101101010011000
110110000101110110011000100110
000000000101011001001001001010
```

main.exe

```
110101011000101100111010100011
010110100100010100111111001100
111010110000010101101010011000
110110000101110110011000100110
000000000101011001001001001010
```

```
110101011000101100111010100011
010110100100010100111111001100
111010110000010101101010011000
110110000101110110011000100110
000000000101011001001001001010
```

```
110101011000101100111010100011
010110100100010100111111001100
111010110000010101101010011000
```

Compilers & Interpreters

Dynamic Linking

Referencing functions that exist within .dll or .so

/lib/libc.so

110101011000101100111010100011

010110100100010100111111001100

111010110000010101101010011000

110101011000101100111010100011

010110100100010100111111001100

111010110000010101101010011000

110110000101110110011000100110

000000000101011001001001001010

110101011000101100111010100011

010110100100010100111111001100

111010110000010101101010011000

110101011000101100111010100011

010110100100010100111111001100

111010110000010101101010011000

main.exe

110101011000101100111010100011

010110100100010100111111001100

111010110000010101101010011000

110110000101110110011000100110

000000000101011001001001001010

110101011000101100111010100011

010110100100010100111111001100

111010110000010101101010011000

110110000101110110011000100110

000000000101011001001001001010

110101011000101100111010100011

010110100100010100111111001100

111010110000010101101010011000

<malloc>



Compilers & Interpreters

Dynamic Linked Libraries (DLLs)

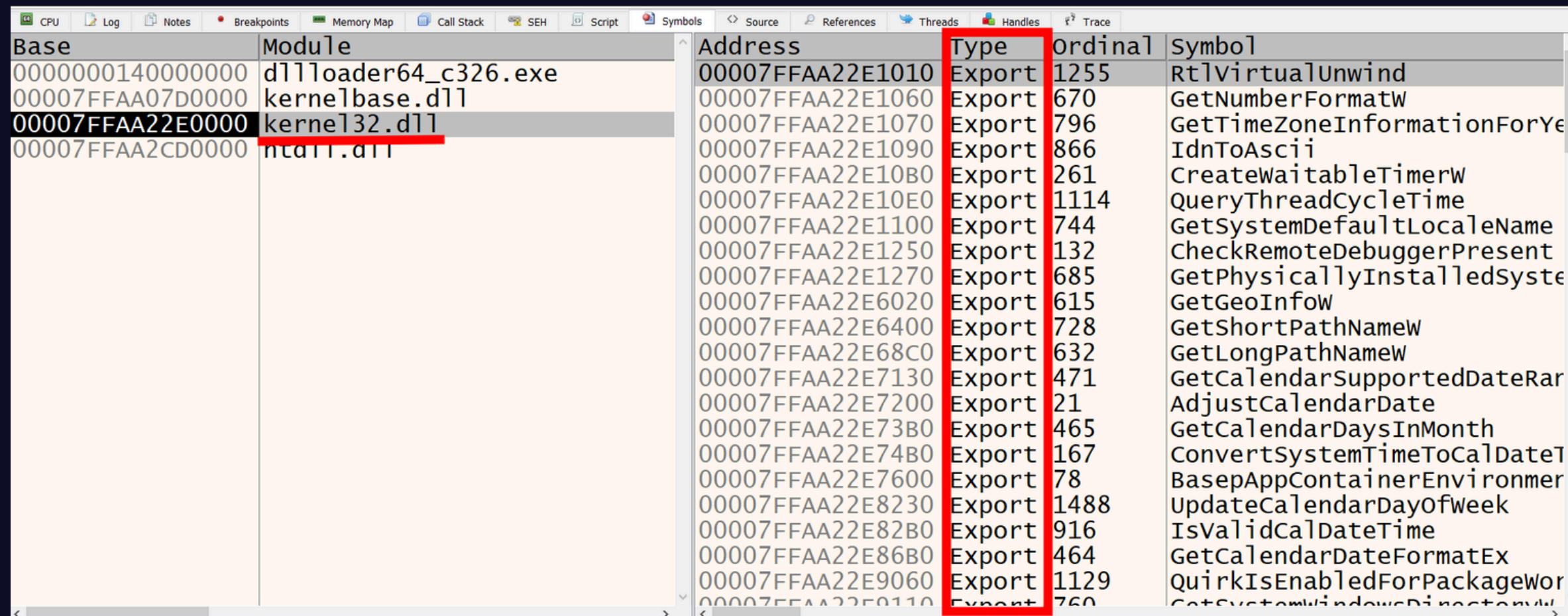
Instead of an executable, compile to a library



Compilers & Interpreters

Dynamic Linked Libraries (DLLs)

A DLL will export all its functions and have no executable code.



Base	Module	Address	Type	Ordinal	Symbol
0000000140000000	dllloader64_c326.exe	00007FFAA22E1010	Export	1255	RtlVirtualUnwind
00007FFAA07D0000	kernelbase.dll	00007FFAA22E1060	Export	670	GetNumberFormatW
00007FFAA22E0000	kernel32.dll	00007FFAA22E1070	Export	796	GetTimeZoneInformationForYe
00007FFAA2CD0000	ntdll.dll	00007FFAA22E1090	Export	866	IdnToAscii
		00007FFAA22E10B0	Export	261	CreateWaitableTimerW
		00007FFAA22E10E0	Export	1114	QueryThreadCycleTime
		00007FFAA22E1100	Export	744	GetSystemDefaultLocaleName
		00007FFAA22E1250	Export	132	CheckRemoteDebuggerPresent
		00007FFAA22E1270	Export	685	GetPhysicallyInstalledSyste
		00007FFAA22E6020	Export	615	GetGeoInfow
		00007FFAA22E6400	Export	728	GetShortPathNameW
		00007FFAA22E68C0	Export	632	GetLongPathNameW
		00007FFAA22E7130	Export	471	GetCalendarSupportedDateRar
		00007FFAA22E7200	Export	21	AdjustCalendarDate
		00007FFAA22E73B0	Export	465	GetCalendarDaysInMonth
		00007FFAA22E74B0	Export	167	ConvertSystemTimeToCalDateT
		00007FFAA22E7600	Export	78	BasepAppContainerEnvironmer
		00007FFAA22E8230	Export	1488	UpdateCalendarDayOfWeek
		00007FFAA22E82B0	Export	916	IsValidCalDateTime
		00007FFAA22E86B0	Export	464	GetCalendarDateFormatEx
		00007FFAA22E9060	Export	1129	QuirkIsEnabledForPackageWor
		00007FFAA22E9110	Export	760	GetSystemWindowsDirectoryW

Code Execution

File Format

Files stored in file-system used formats to help in loading



Code Execution

Structured File Formats:

PE:



- For Windows
- Used in
 - .exe
 - .dll

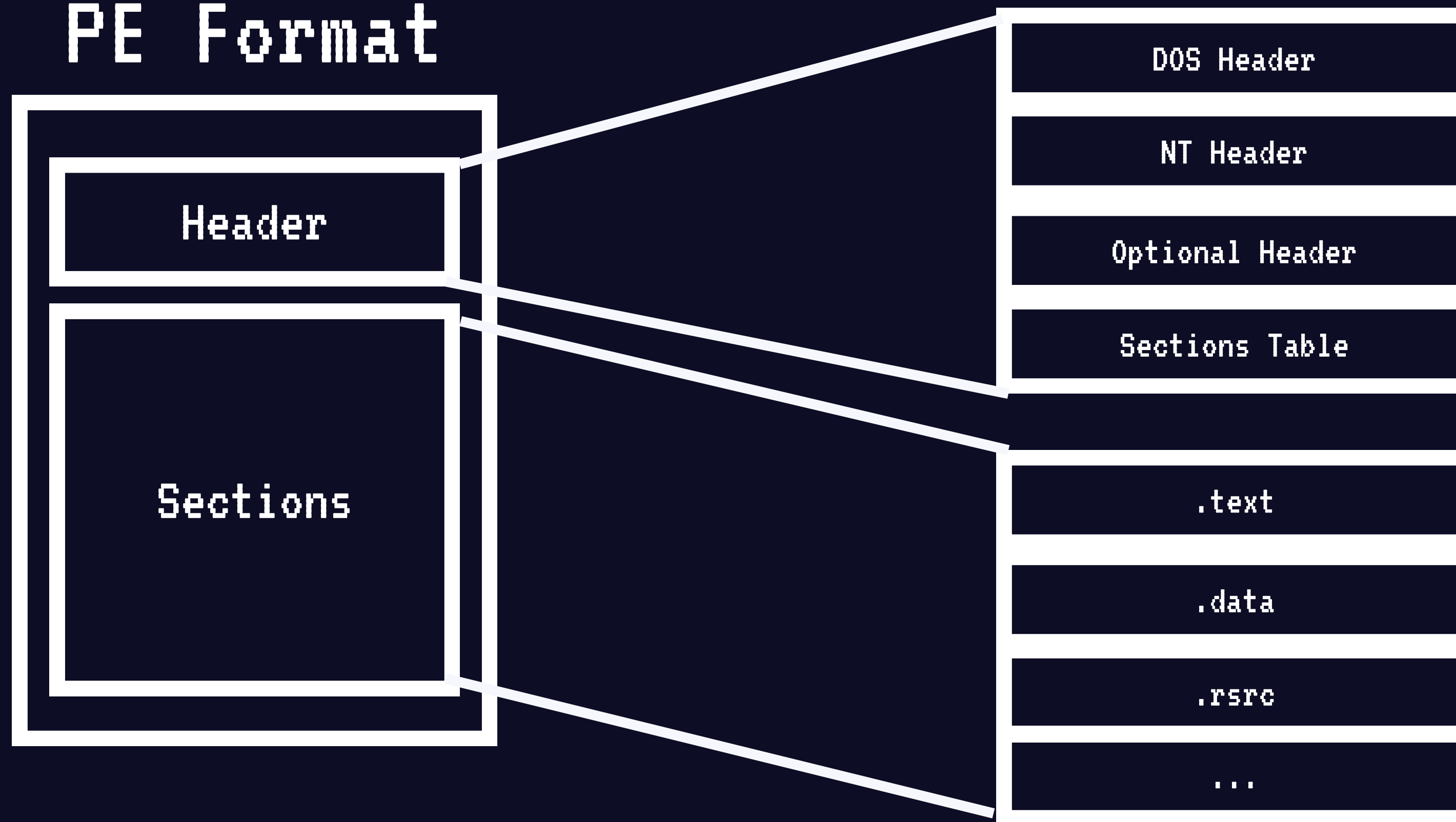
ELF:



- For Linux
- Used in
 - .bin
 - .so

Code Execution

PE Format



Code Execution

PE headers can reveal a lot of information
for malware analysts

- Imports
- Section sizes (useful to tell if packed)
- CPU Type
- Constant section data
 - .rdata (constants)
 - .bss (uninitialized vars)
 - .rsrc (resource files)

Code Execution

```
~/Downloads/myfiles  
(18:09:53) → ls  
bingus interp.exe out.sprx  
~/Downloads/myfiles  
(18:09:53) → file out.sprx  
out.sprx: data  
~/Downloads/myfiles  
(18:09:55) → file interp.exe  
interp.exe: PE32+ executable for MS Windows 6.00 (console), x86-64, 6  
~/Downloads/myfiles  
(18:09:58) → file bingus  
bingus: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynam  
ID[sha1]=c7273306d5bd5349517eac1fec8e7076774f3cd2, for GNU/Linux 3.2.0
```

Linux **file** command can view file types

Code Execution

File on disk has no idea where it is loaded, so we describe addresses through **Relative Virtual Addresses (RVA)** from an assumed **Base Address**

Base Addr = 0x400000

* Convention for Windows

$RVA = Base\ Addr + Offset$

Code Execution

RVA

```
00401060 void _start(int64_t arg1, int64_t arg2, int64_t arg3)
00401060 31ed xor ebp, ebp {0x0}
00401062 4989d1 mov r9, rdx
00401065 5e pop rsi {__return_addr}
00401066 4889e2 mov rdx, rsp {arg_8}
00401069 4883e4f0 and rsp, 0xfffffffffffff0
0040106d 50 push rax {var_8}
0040106e 54 push rsp {var_8} {var_10}
0040106f 4531c0 xor r8d, r8d {0x0}
00401072 31c9 xor ecx, ecx {0x0}
00401074 488d3dce000000 lea rdi, [rel sub_401149]
0040107b ff153f2f0000 call qword [rel __libc_start_main]
00401081 f4 hlt
{ Does not return }
```


Code Execution

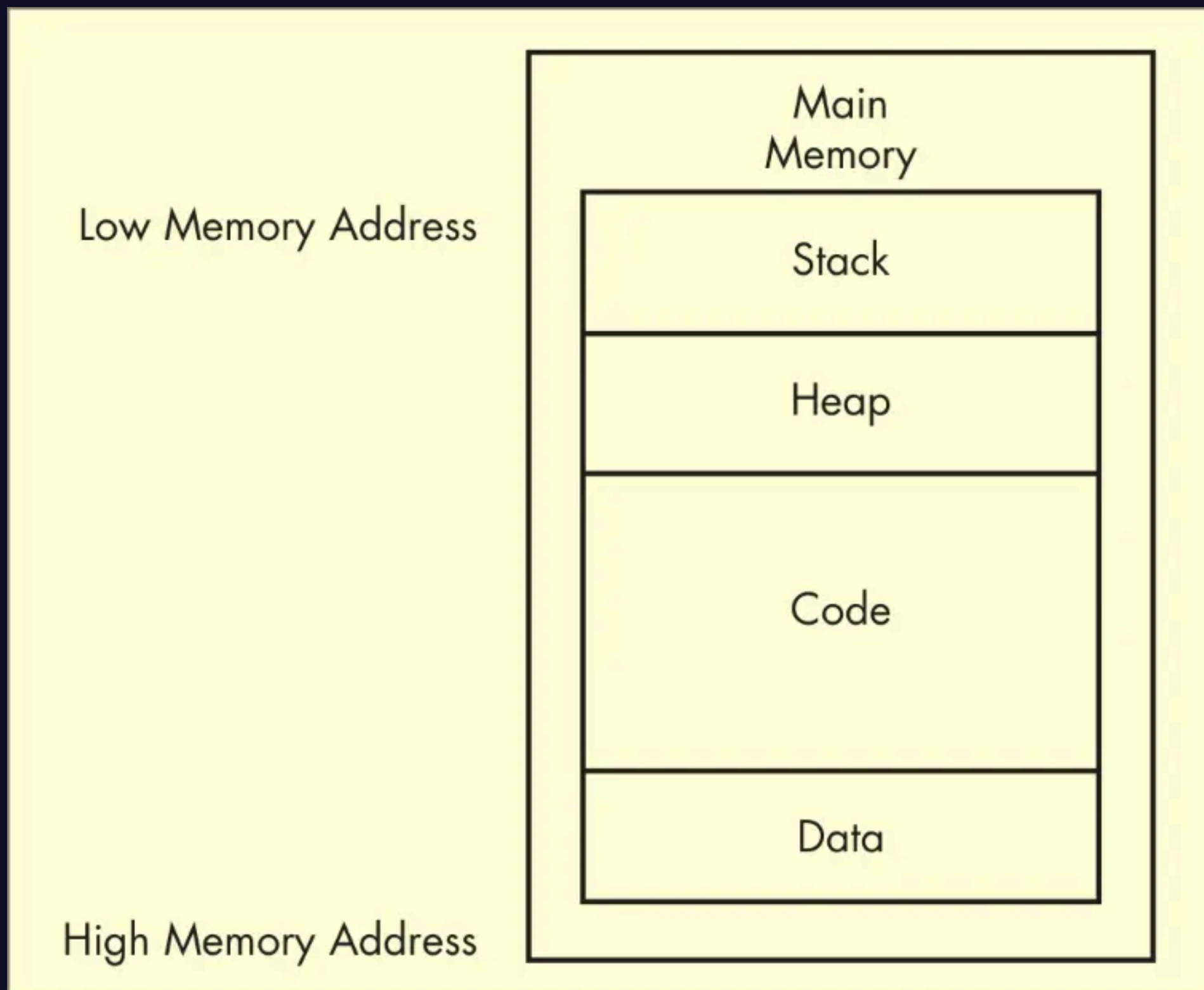
Once file loaded into RAM, OS determines the **Base Address** through **Address Space Layout Randomization**

* If ASLR is disabled, it loads at base address defined in program header

```
► 0x7fb275e276a0 <__libc_start_main>      endbr64
0x7fb275e276a4 <__libc_start_main+4>    push    rbp
0x7fb275e276a5 <__libc_start_main+5>    mov     rbp, rsp
```

```
► 0x7f8700c276a0 <__libc_start_main>      endbr64
0x7f8700c276a4 <__libc_start_main+4>    push    rbp
0x7f8700c276a5 <__libc_start_main+5>    mov     rbp, rsp
```

Code Execution

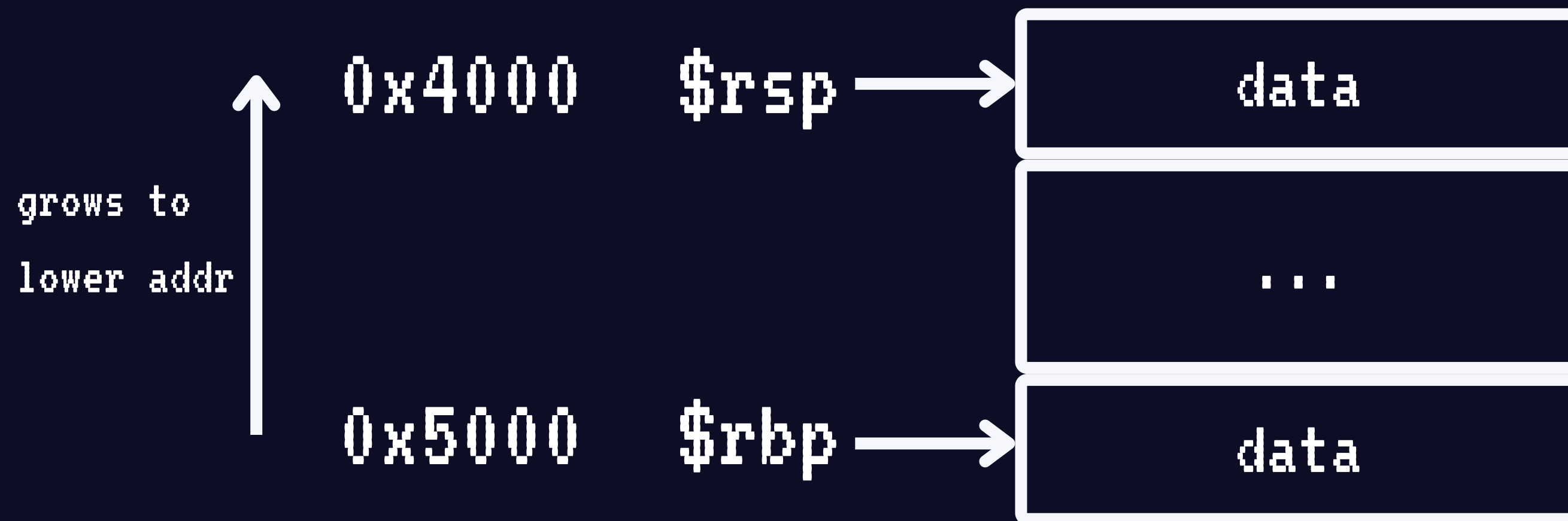


Program being loaded into RAM

Code Execution

Stack

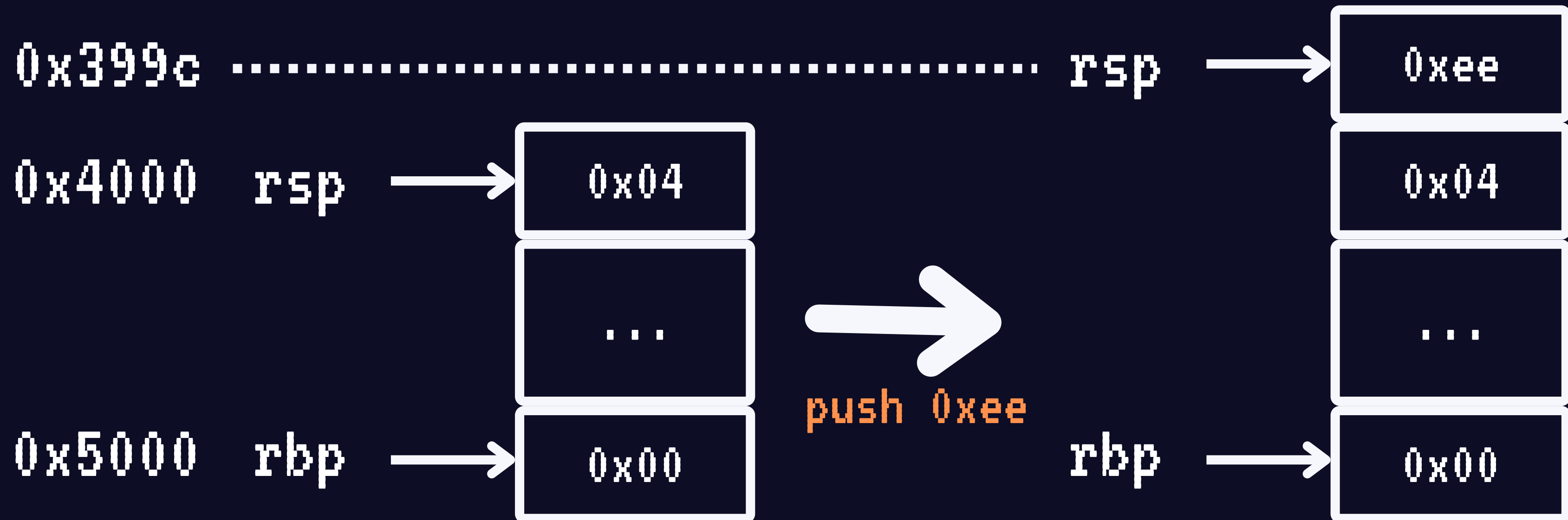
A 1-8MB storage space in the program memory used for variables that can't be stored in registers



Code Execution

Push

1. `sub rsp, <bytesize of data>`
2. move new data at `rsp`



Code Execution

Pop

1. add rsp, <bytesize of data>

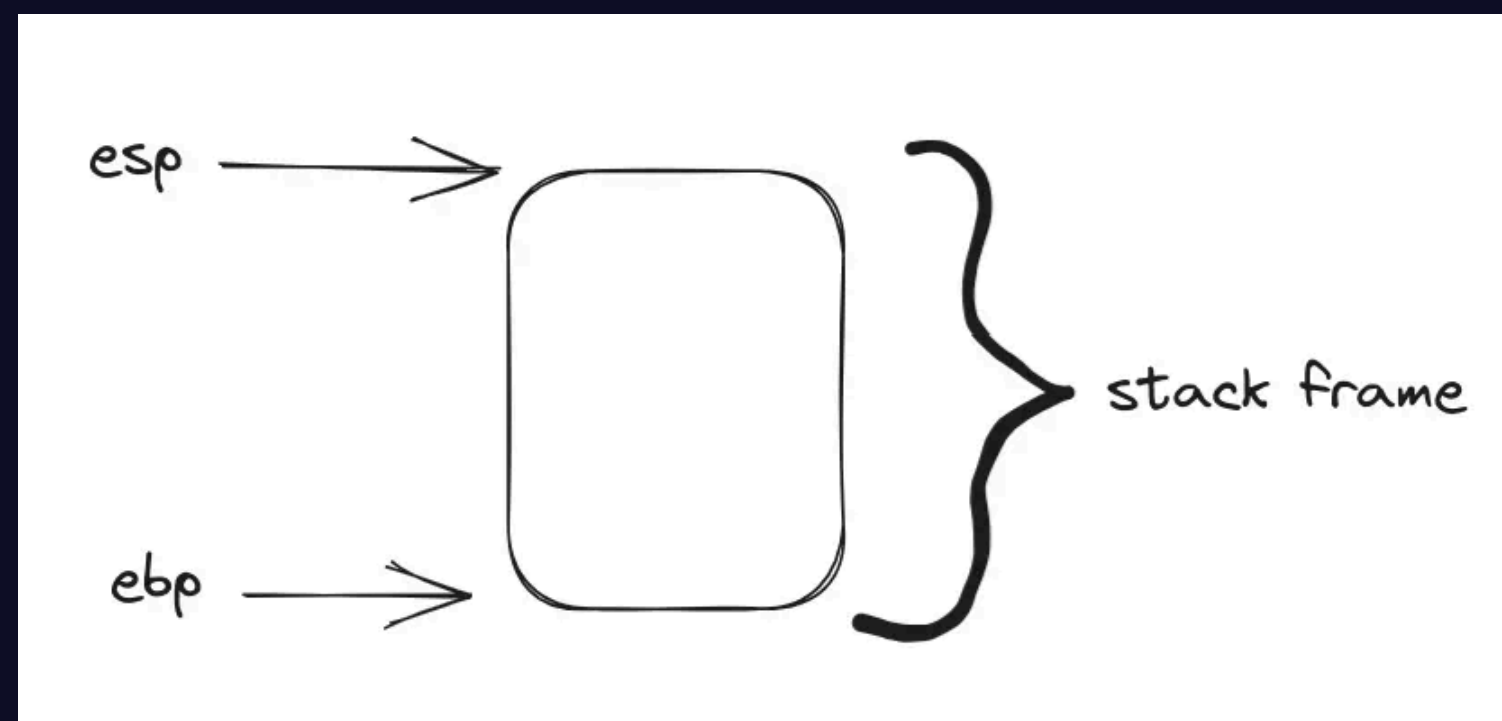


Code Execution

Functions/Stack Frames

A functions scope is defined by:

- rbp (base pointer register)
- rsp (stack pointer register)



local variables are accessed by offset from rsp or rbp

Code Execution

Calling conventions:

When functions are called we establish:

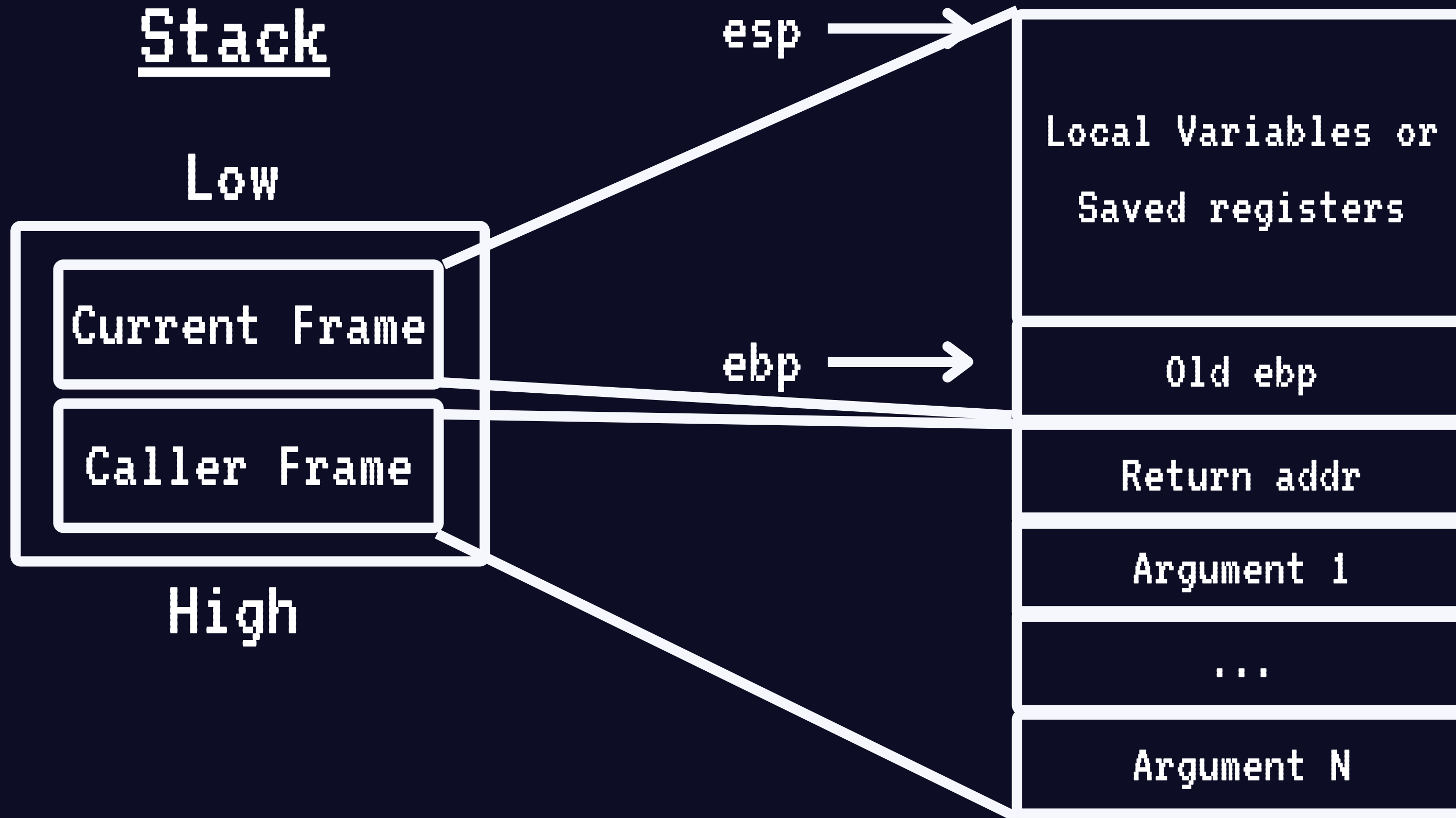
- How arguments are passed
- How stack is cleaned after function-run
- What registers are used (if any)

Code Execution

x86 Function Call Process (Win32 stdcall):

1. Push all arguments onto stack
2. call <function_address> & push rip
3. push rbp
4. Allocate stack space for new local variables/saved regs
5. Do Function logic
6. Free up stack space of local variables/saved regs
7. pop rbp
8. ret N (callee cleans up arguments)

Code Execution

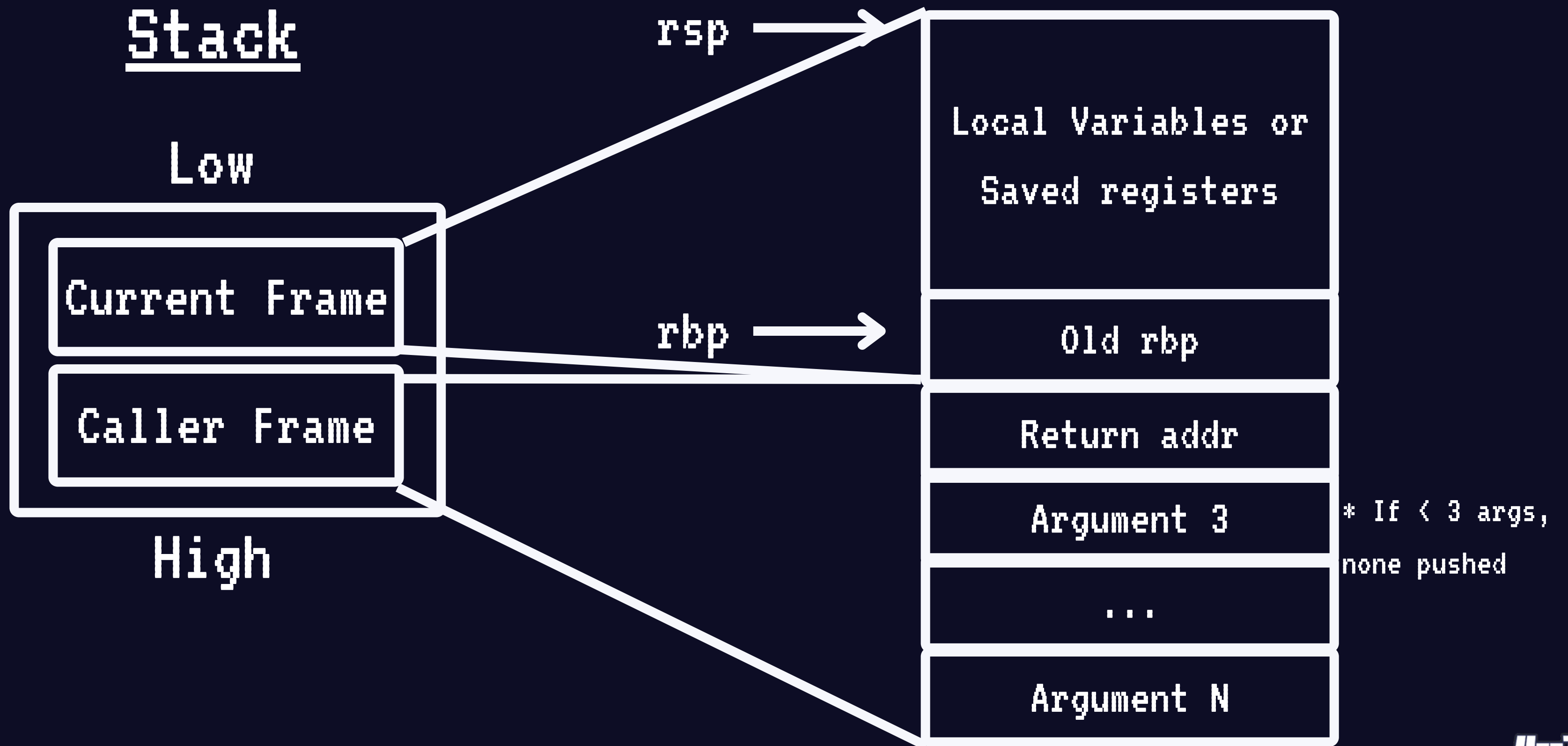


Code Execution

x86 Function Call Process (fastcall):

1. Store first 2 arguments into registers edx, ecx
2. Push all subsequent arguments (if any)
 3. call <function_address> & push rip
 4. push rbp
 5. Allocate stack space for new local variables/saved regs
 6. Do Function logic
 7. Free up stack space of local variables/saved regs
8. pop rbp
9. ret N (callee cleans up arguments)

Code Execution

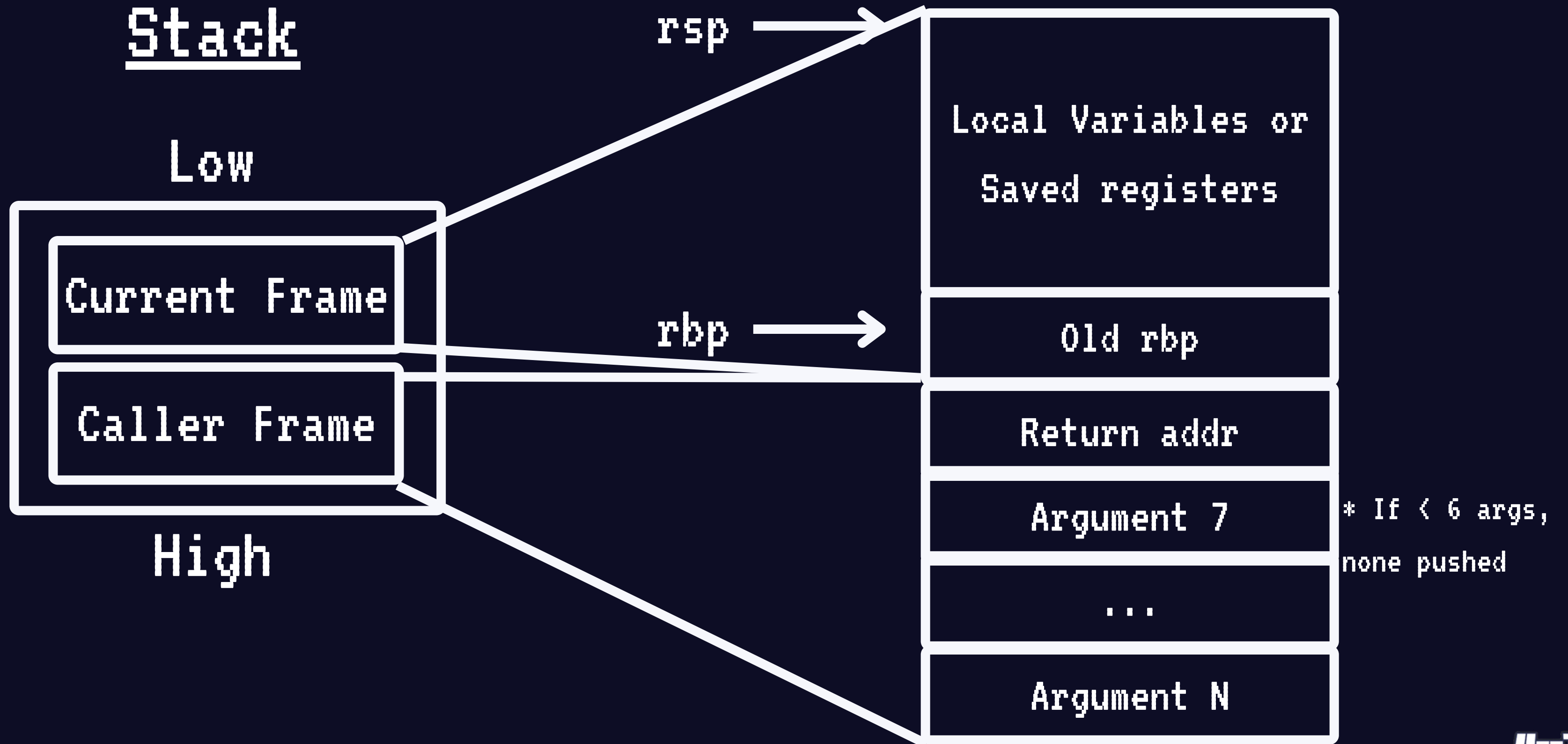


Code Execution

x86 Function Call Process (Unix/SysV):

1. Store at most 6 arguments into general purpose registers
2. Push all subsequent arguments (if any)
3. call <function_address> & push rip
4. push rbp
5. Allocate stack space for new local variables/saved regs
6. Do Function logic
7. Free up stack space of locals/saved regs
8. pop rbp
9. ret
10. Caller cleans up stack space

Code Execution



Code Execution

Constants / Data section

Constants live in the **data section**, their addresses are constant after link-time

Constants are accessed via **rip + offset**

```
int myglob = 0xFF;  
  
int main() {  
    int h = myglob;  
}
```



```
mov     eax, dword ptr [rip + 0x2eed]
```

Code Execution

Endness

Big Endian : Values stored most -> least significant bit

- Data over network



Little Endian : Values stored least -> most significant bit

- Data stored in memory



10 Min Break

Optional:

If you want to download PwnDBG and this binary:

<https://tinyurl.com/2yrnbs4j>

We will cover:

4. Decompilers

5. Debuggers

6. Obfuscation

7. Symbolic Execution

Decompiling

Process of converting a binary back into source code



Ghidra

Free



Binja

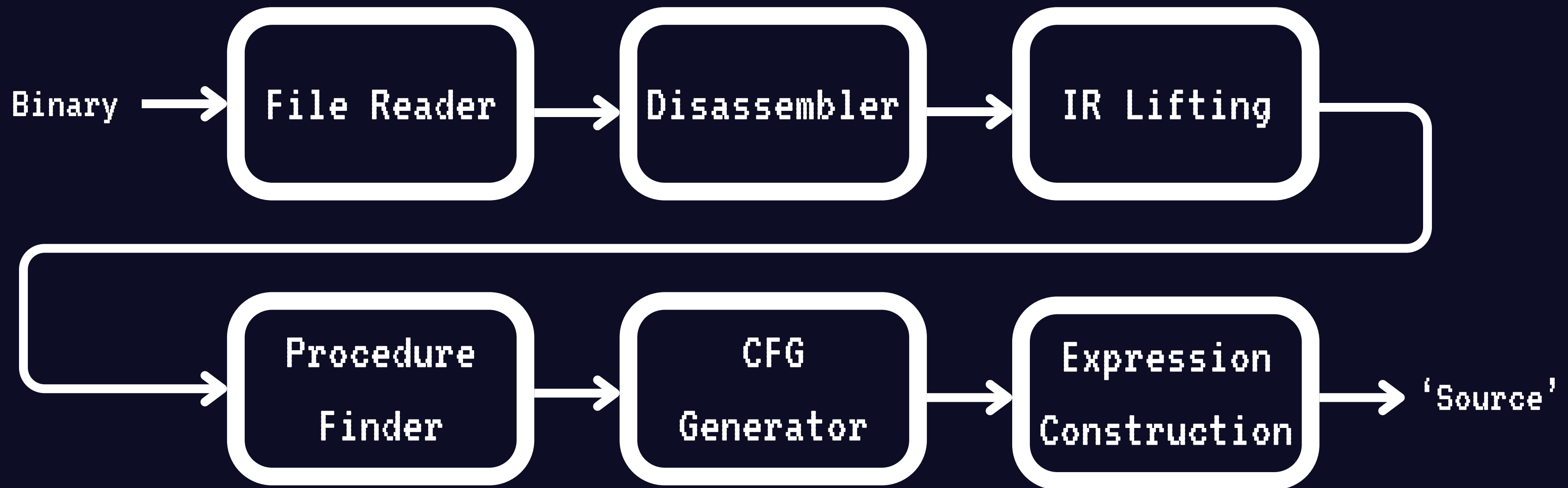


Hexrays

Paid

Decompiling

Decompilation Process



Decompiling

File Reader:

A program that reads data from a file format:

- Global variables
- Global functions
- Addresses and names of imported functions
- Address of the start of instructions

Decompiling

Disassemblers

Tools to convert opcodes back into assembly

Modern disassemblers are very robust, but
can still be prone to errors

[illegible]

```
mov r12, rax
```

```
add rbx, 0x5
```

```
test edx, 0x2
```


Decompiling

IR Lifting

Tools to convert assembly into a platform independent IR

VexIR can convert many different ISA

```
                                t0 = GET:I32(16)
                                t1 = 0x8:I32
subs R2, R2, #8  →  t3 = Sub32(t0,t1)
                                PUT(16) = t3
                                PUT(68) = 0x59FC8:I32
```

Decompiling

Code Walker / Procedure Finder

Static analysis tool to find all functions within a binary.

Performed by viewing all:

- Jump instructions
- Call instructions
- Instructions commonly associated with stack frame

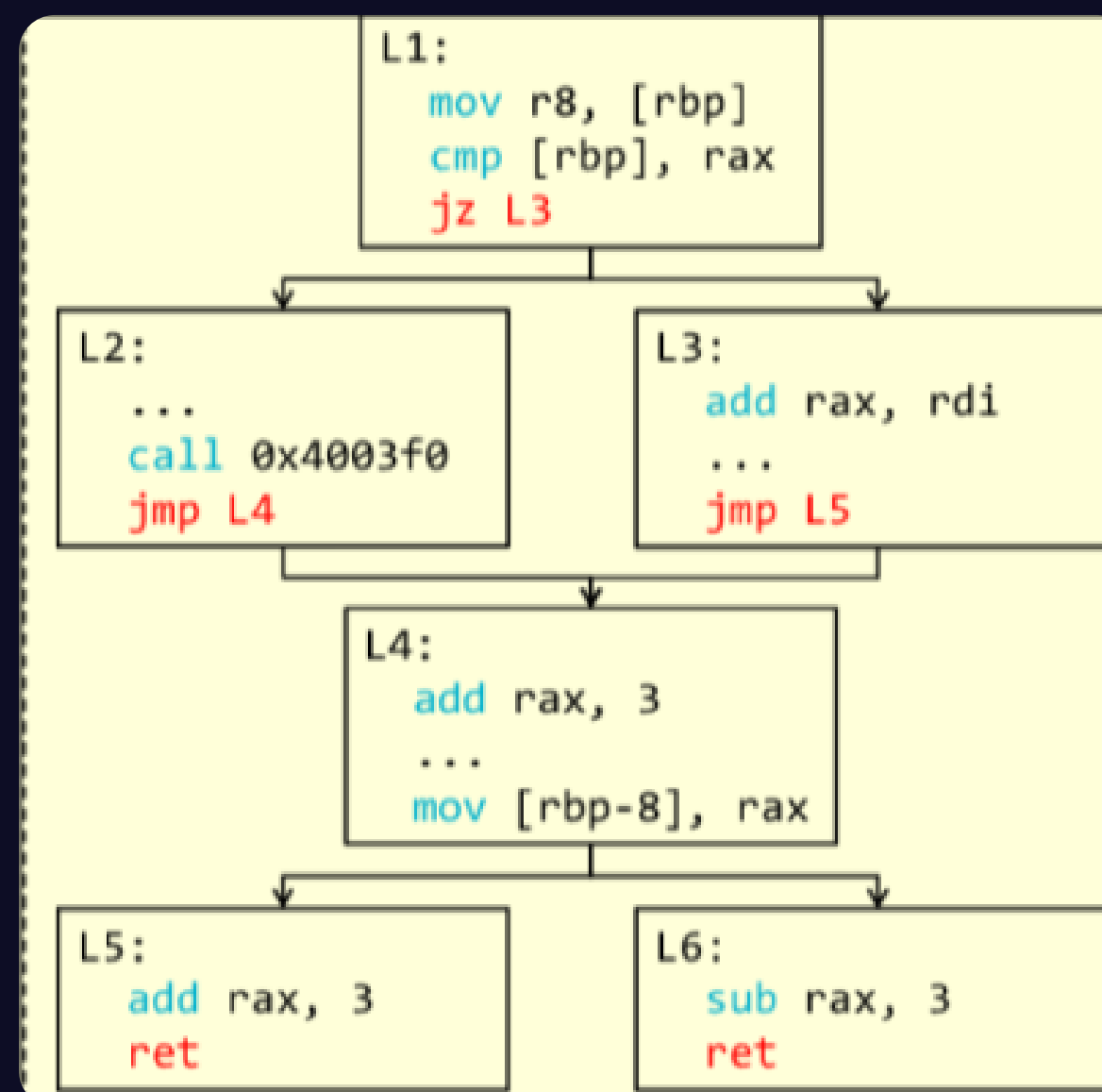
Decompiling

Control Flow Graph Generator

A tool used to find control flow of basic blocks

Basic Block :

sequence of
instructions executed
linearly with one
entry and one exit



Decompiling

Expression Generation

The process that:

- Recovers types
- Simplifies conditionals
- Creates loops
- Convert registers to variables
- Generates functions

Debugging

Tools to trace a program during execution

Userland Debuggers:

Used for apps

- x32dbg (windows)
- gdb (linux)

Kernel Debuggers:

Used for drivers & OS

- WinDBG (windows)

Debugging

gdb is able to:

- **step** to the next instruction
- **step into** a function
- **print** local information
- **examine** memory information
- set a **breakpoint** for the current instruction
- set a **watchpoint** for variable changes
- **continue** to the next breakpoint

Debugging

We are using the gdb extension pwndbg.



```
*RAX 0x7fffffffefa90 ← 0x72756f206f686365 ('echo our')
RBX 0x0
*RCX 0x7ffff7b165a0 (__write_nocancel+7) ← cmp rax, -0xfff
*RDX 0x7ffff7dd5770 (_IO_stdfile_0_lock) ← 0x0
*RDI 0x7fffffffefa90 ← 0x72756f206f686365 ('echo our')
*RSI 0x1
*R8 0x602000 ← 0x0
*R9 0x26
*R10 0x56
*R11 0x246
*R12 0x4006b0 ← xor ebp, ebp
*R13 0x7fffffffec80 ← 0x2
R14 0x0
R15 0x0
*RBP 0x7fffffffefba0 → 0x400950 ← push r15
*RSP 0x7fffffffefa80 → 0x7fffffffec88 → 0x7fffffffefee6b ← 0x2f6465726168532f ('/Shared/')
*RIP 0x400862 ← call 0x400630
```

It can be installed here:

<https://pwndbg.re/pwndbg/latest/setup/>

Debugging

Lets debug this program in GDB

```
int main(int argc, char *argv[]){  
    char myflag[] = "flag{secret}";  
    for (int i = 0; i < strlen(myflag); i++){  
        if (argv[1][i] != myflag[i]) return 0;  
    }  
    win();  
    return 0;  
}  
  
void win(){ printf("correct"); }
```

```
gcc myprog.c -Wall -o myprog && strip --strip-all myprog
```


Debugging

```
'gdb ./myprog'
```

```
To make this setting permanent, add 'set debug  
(No debugging symbols found in ./myprog)
```

```
----- tip of the day (disable with set show-  
stepuntilasm <assembly-instruction [operands]>  
pwndbg> █
```

Debug symbols:

Data added to a program by compilers for debuggers to have full information about source code. GCC's '-g' flag

Debugging

'break main'

```
pwndbg> b main  
Function "main" not defined.
```

*most C programs have main, but this symbol is not defined

Stripped Binaries:

Binaries that have their symbols and metadata removed.

Done with the linux `strip` command

Debugging

'starti'

```
► 0x7ffff7fe0b40  mov    rdi, rsp      RDI => 0x7ffffffffffd7a0 ← 1
0x7ffff7fe0b43  call   0x7ffff7fe1690 <0x7ffff7fe1690>

0x7ffff7fe0b48  mov    r12, rax
0x7ffff7fe0b4b  mov    r13, rsp
0x7ffff7fe0b4e  mov    edx, dword ptr [rip + 0x1cc5c]    EDX, [_rtld_global+1968]
0x7ffff7fe0b54  test   edx, 2
0x7ffff7fe0b5a  je     0x7ffff7fe0b6d <0x7ffff7fe0b6d>
```

* We are in the linux program loader right now

We have to find main manually.

First, we need to load the process into RAM.

Lets run and break at the first instruction

Debugging

'info files'

```
pwndbg> info files
Symbols from "/home/david/Documents/C/myprog".
Native process:
    Using the running image of child process 110297.
    While running this, GDB does not access memory from...
Local exec file:
    ` /home/david/Documents/C/myprog', file type elf64-x86-64.
Entry point: 0x555555555060
0x00005555555554350 - 0x00005555555554390 is .note.gnu.property
0x00005555555554390 - 0x000055555555543b4 is .note.gnu.build-id
0x000055555555543b4 - 0x000055555555543d0 is .interp
0x000055555555543d0 - 0x000055555555543ec is .gnu.hash
0x000055555555543f0 - 0x00005555555554408 is .dynamic
```

This is the entry point of our C program

Debugging

'x/16i 0x5555555555060'

```
pwndbg> x/16i 0x5555555555060
0x5555555555060:      endbr64
0x5555555555064:      xor     ebp,ebp
0x5555555555066:      mov     r9,rdx
0x5555555555069:      pop     rsi
0x555555555506a:      mov     rdx,rsp
0x555555555506d:      and     rsp,0xfffffffffffffffff0
0x5555555555071:      push    rax
0x5555555555072:      push    rsp
0x5555555555073:      xor     r8d,r8d
0x5555555555076:      xor     ecx,ecx
0x5555555555078:      lea     rdi,[rip+0xf5]          # 0x5555555555174
0x555555555507f:      call    QWORD PTR [rip+0x2f3b] # 0x55555555557fc0
0x5555555555085:      hlt
```

← call _libc_start_main(mainaddress, ...)

The first argument to _libc_start_main is our main function

Debugging

```
'b *(0x5555555555174)'
```

```
pwndbg> b *(0x5555555555174)
```

```
Breakpoint 1 at 0x5555555555174
```

```
pwndbg> info break
```

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	y	0x00005555555555174	

We've set a break point at main so we can stop at the main function

Debugging

```
'b *(0x5555555555174)'
```

```
pwndbg> b *(0x5555555555174)
```

```
Breakpoint 1 at 0x5555555555174
```

```
pwndbg> info break
```

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep y		0x00005555555555174	

We've set a break point at main so we can stop at the main function

Debugging

'continue'

```
[ DISASM / x86-64 / set emulate on ]
► 0x555555555174    push    rbp
0x555555555175    mov     rbp, rsp                RBP => 0x7fffffffffd680 -> 0x7
0x555555555178    push    rbx
0x555555555179    sub     rsp, 0x38              RSP => 0x7fffffffffd640 (0x7ff
0x55555555517d    mov     dword ptr [rbp - 0x34], edi [0x7fffffffffd64c] <= 1
0x555555555180    mov     qword ptr [rbp - 0x40], rsi [0x7fffffffffd640] <= 0x7fffff
cuments/C/myprog'
0x555555555184    mov     rax, qword ptr fs:[0x28] RAX, [0x7ffff7f78768] => 0x4
0x55555555518d    mov     qword ptr [rbp - 0x18], rax [0x7fffffffffd668] <= 0x4f1eb7
0x555555555191    xor     eax, eax               EAX => 0
0x555555555193    movabs  rax, 0x6365737b67616c66 RAX => 0x6365737b67616c66 (
0x55555555519d    mov     qword ptr [rbp - 0x25], rax [0x7fffffffffd65b] <= 0x636573
```

We continue, until we hit the breakpoint.

Now we are in main, lets re-run with arguments

Debugging

```
'r flag{secret}'
```

0x5555555555174	push	rbp	reset and allocate space on stack
0x5555555555175	mov	rbp, rsp	
0x5555555555178	push	rbx	
0x5555555555179	sub	rsp, 0x38	store argc, argv on stack
0x555555555517d	mov	dword ptr [rbp - 0x34], edi	
0x5555555555180	mov	qword ptr [rbp - 0x40], rsi	
ents/C/myprog'			generate stack canary and store on stack
0x5555555555184	mov	rax, qword ptr fs:[0x28]	
0x555555555518d	mov	qword ptr [rbp - 0x18], rax	set eax = 0
0x5555555555191	xor	eax, eax	

You will see this structure for all C main()

Debugging

'ni' (like 20 times)

```
0x5555555555191  xor     eax, eax
0x5555555555193  movabs  rax, 0x6365737b67616c66
0x555555555519d  mov     qword ptr [rbp - 0x25], rax
0x55555555551a1  movabs  rax, 0x7d746572636573
0x55555555551ab  mov     qword ptr [rbp - 0x20], rax
0x55555555551af  mov     dword ptr [rbp - 0x2c], 0
0x55555555551b6  jmp     0x55555555551e7
```

store the string
"flag{secret}" onto stack
for loop.
new variable i = 0, store
on stack
then, we jump into a loop

Debugging

'ni' (like 8 times)

0x5555555551af	mov	dword ptr [rbp - 0x2c], 0	
0x5555555551b6	jmp	0x5555555551e7	<
↓			
0x5555555551e7	mov	eax, dword ptr [rbp - 0x2c]	] move the value of i into rbx
0x5555555551ea	movsxd	rbx, eax	
0x5555555551ed	lea	rax, [rbp - 0x25]	] rax = strlen(myflag)
▶ 0x5555555551f1	mov	rdi, rax	
0x5555555551f4	call	strlen@plt	<
0x5555555551f9	cmp	rbx, rax	] do a loop while i < flag
0x5555555551fc	jb	0x5555555551b8	
0x5555555551fe	call	0x555555555159	<
0x555555555203	mov	eax, 0	EAX =>

Debugging

```
'b *(0x55555555551f9)'
```

```
► 0x5555555551f1  mov    rdi, rax
0x5555555551f4  call   strlen@plt

0x5555555551f9  cmp    rbx, rax
0x5555555551fc  jb     0x5555555551b8
```

Set a breakpoint at this compare so we
don't need to 'ni' several times

```
pwndbg> b *(0x55555555551f9)
```

```
Breakpoint 2 at 0x55555555551f9
```

```
pwndbg> info break
```

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep y		0x0000555555555174	
	breakpoint already hit 1 time				
2	breakpoint	keep y		0x00005555555551f9	

Debugging

'c' (about 13 times idk)

```
► 0x5555555551f9    cmp    rbx, rax
    0x5555555551fc    jb     0x5555555551b8
    0x5555555551fe    call  0x555555555159
```

```
pwndbg> p $rbx
$3 = 12
pwndbg> p $rax
$4 = 12
```

Continue until the loop is over.

It ends when \$rbx = \$rax

Debugging

'ni' (2 times)

```
b+ 0x55555555551f9      cmp     rbx, rax
    0x55555555551fc      jb      0x55555555551b8

▶ 0x55555555551fe      call   0x5555555555159
```

now, we are going to call the win() function at this address

Debugging

'si'

▶ 0x5555555555159	push	rbp	] reset the stack for the current function
0x555555555515a	mov	rbp, rsp	
..			
0x555555555515d	lea	rax, [rip + 0xea0]	] move the string "correct" into rax, then call printf
0x5555555555164	mov	rdi, rax	
0x5555555555167	mov	eax, 0	
0x555555555516c	call	printf@plt	
0x5555555555171	nop		] return back to main()
0x5555555555172	pop	rbp	
0x5555555555173	ret		

We step inside of the function

Debugging

'ni' (like 8 times)

0x5555555521c	mov	rbx, qword ptr [rbp - 8]	
0x55555555220	leave		
► 0x55555555221	ret		
↓			
0x7ffff7c27675	mov	edi, eax	
0x7ffff7c27677	call	exit	

leave main
then end the program

ok! we have traced through a basic program!

Obfuscation

The practice of making source code or assembly unreadable to humans and tools:

- Source-Level obfuscation
- Binary-Level obfuscation
- Packers

Obfuscation

Source-level obfuscation

- Changing variable names
- Inserting meaningless code
- Mangling conditional statements
- Putting code in eval()
- Encrypting literals

Obfuscation

Source-level obfuscation

But WHY?

- Some languages like Javascript require being ran on the client-side browser
- Some languages like dotNET require bytecode to be visible to client

Obfuscation

Obfuscation Tools

- JSFuck
- JSCrambler
- Dotfuscator

Obfuscation

Binary Obfuscation - Encrypting Literals

Strings and function names can be encrypted so direct references do not make sense.

```
encryptedString = "\uBDC6\uFCDE\u3C87\u7C97\uBC4F\uFE57\u3EE8\u7C98" +  
....."\uBCE0\uFC48\u3E19\u7C59\uBC51\uFF01\u3EEA\u7DCA" +  
....."\uBEC2";
```

Obfuscation

Binary Obfuscation - Function Pointers

Instead of calling the function itself, we call a trampoline function.

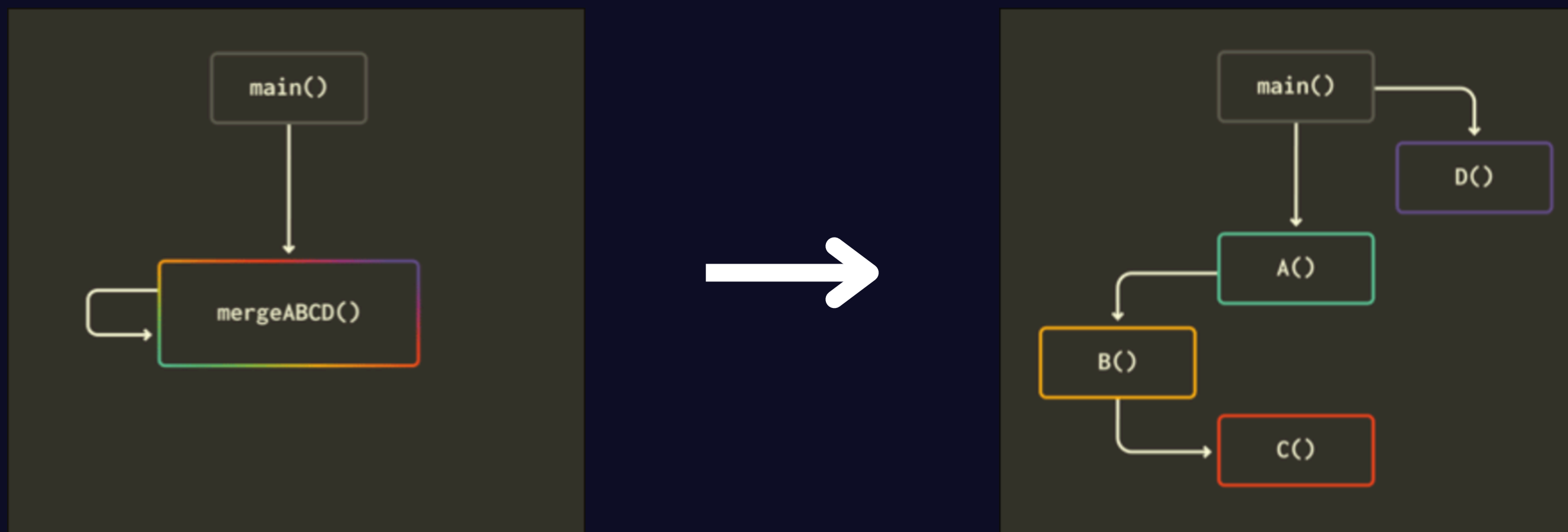
The trampoline computes a function pointer to call.

ObfVirtualAlloc() → Trampoline() → VirtualAlloc()

Obfuscation

Binary Obfuscation - Control Flow Obfuscation

Instead of a single function, have many unconditional calls to smaller functions



Obfuscation

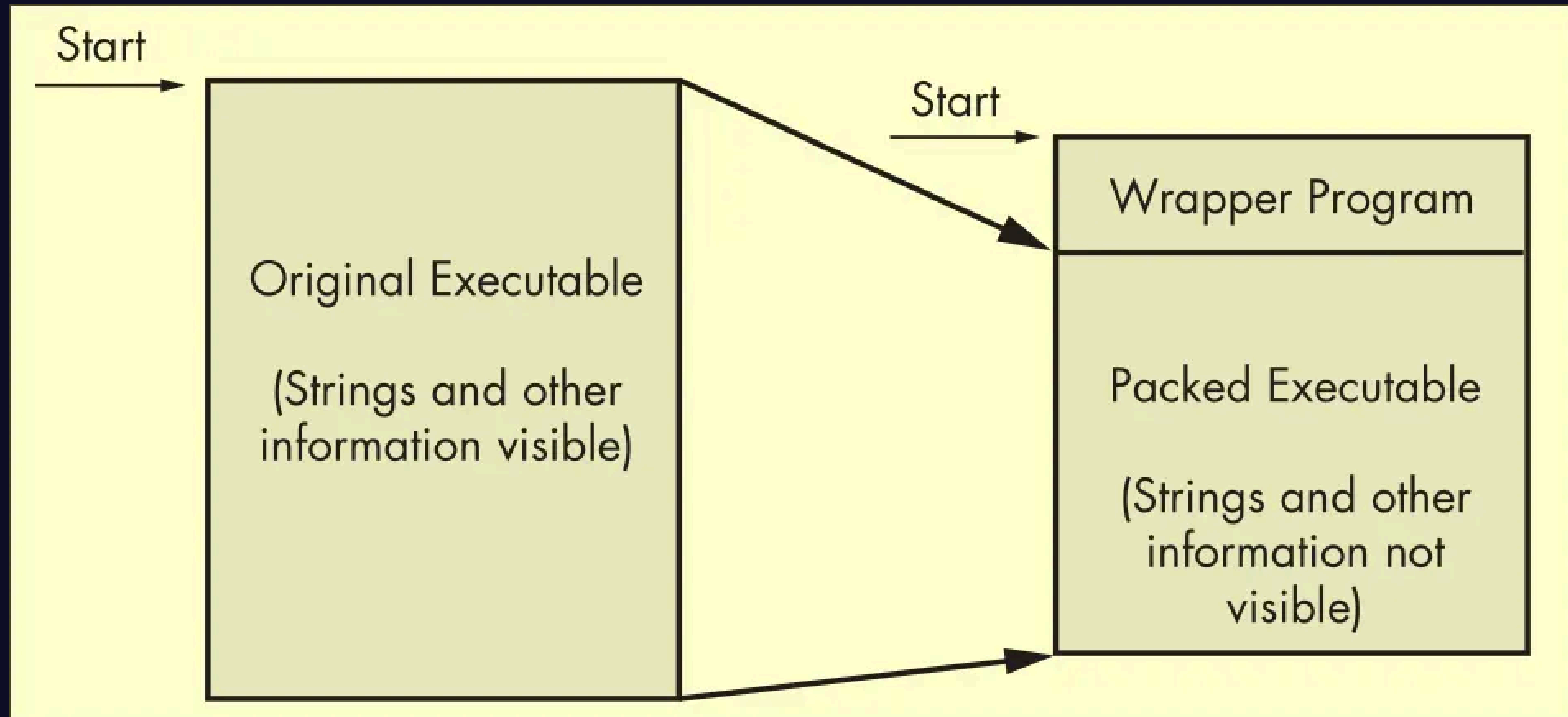
Binary Obfuscation - Mixed Boolean Arithmetic

Convert simple boolean expressions into convoluted ones

```
int what(int x, int y, int z, int w) {  
    return 4071272 * w + 3590309086 * (w | z & z & (y |  
        w)) + 3690425492 * (w & x ^ ~z ^ ~y) + 3735539420 *  
        (y ^ (w & z | y)) + 3176111544 * ((x & y | x ^ z) ^  
        ~y) + 90227856 * (y & x & x & (x & w | ~w)) +  
        2231609705 * (~z ^ w & x | z ^ x | x | w) +  
        263864489 * (w ^ z | x | w | z) + 17029904 *  
        (~w ^ w ^ (x | z) ^ ~x) + 2987805917 * (z & x) +  
        1280785607 * z + 2092481170 * (y & (w & y | ~z)) +  
        637019375 * (~w | w & z & ~x) + 3221225472 * ((x |  
        w) ^ x ^ x ^ y) + 3985988880 * x + 263864489 * (~w &  
        x) + 469200020 * ((z ^ w & w) & ~(x ^ y)) +  
        1774328762 * ((w | x) & (x ^ z) & z) + 3645311564 *  
        ~(z | w) | w) + 3194849700 * ~((y | y) ^ y ^ z)  
        + 1678283628 * ~(~y & ~w) + 1083630375 * y;  
}
```


Obfuscation

Packers

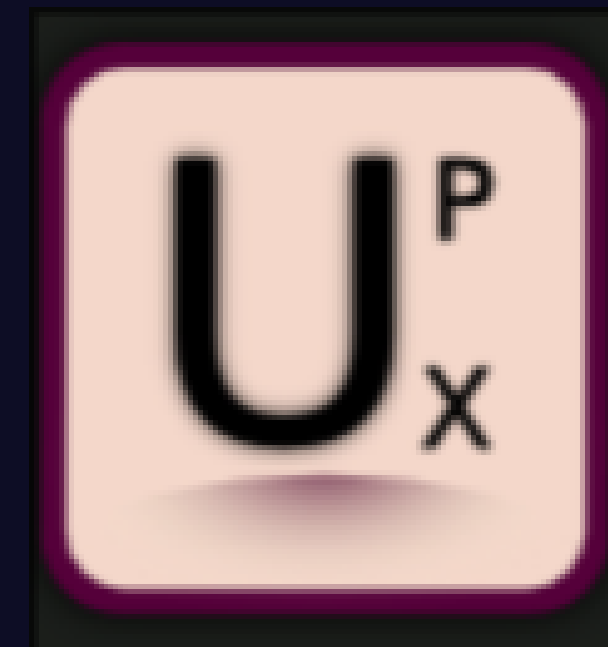


Obfuscation

Packers - Ultimate Packer (UPX)

A simple packer used to reduce file size
with the UCL compression algorithm

On executable run, it runs
the decompression software



Obfuscation

Packers - VM Protect (VMP)

A packer that converts binary code into custom bytecode targetted for a custom virtual machine

* Virtual machines are different for every program



Obfuscation

Packers - Themida

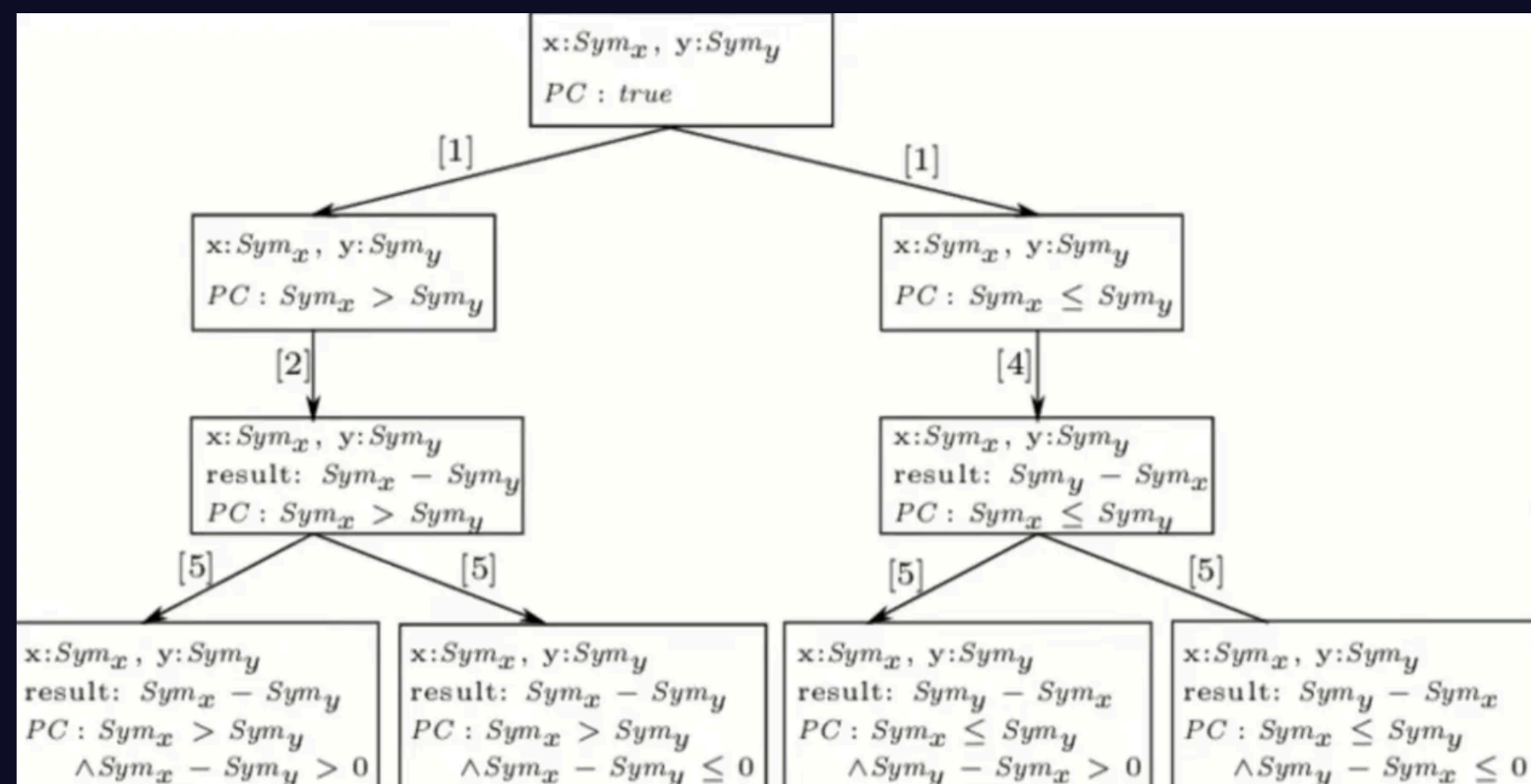
A packer that uses SecureEngine that comes with:

- Import table obfuscation
- Anti-debugging
- Code encryption
- VM-based packing like VMP



Symbolic Execution

Constructing a program as a series of constraints then using solvers to find inputs that solve the constraints



Symbolic Execution

Constraint:

This is a specific condition imposed on a variable as a result of reaching some basic block

- $x > 5$
- $x * 4 == 20$
- $y - 3 \geq 6$

Symbolic Execution

Symbolic Variables:

These are variables with unknown values at runtime, they exist to satisfy constraints.

- Int
- Bitvector
- Real

Symbolic Execution

Satisfiability Modulo Theories (SMT):

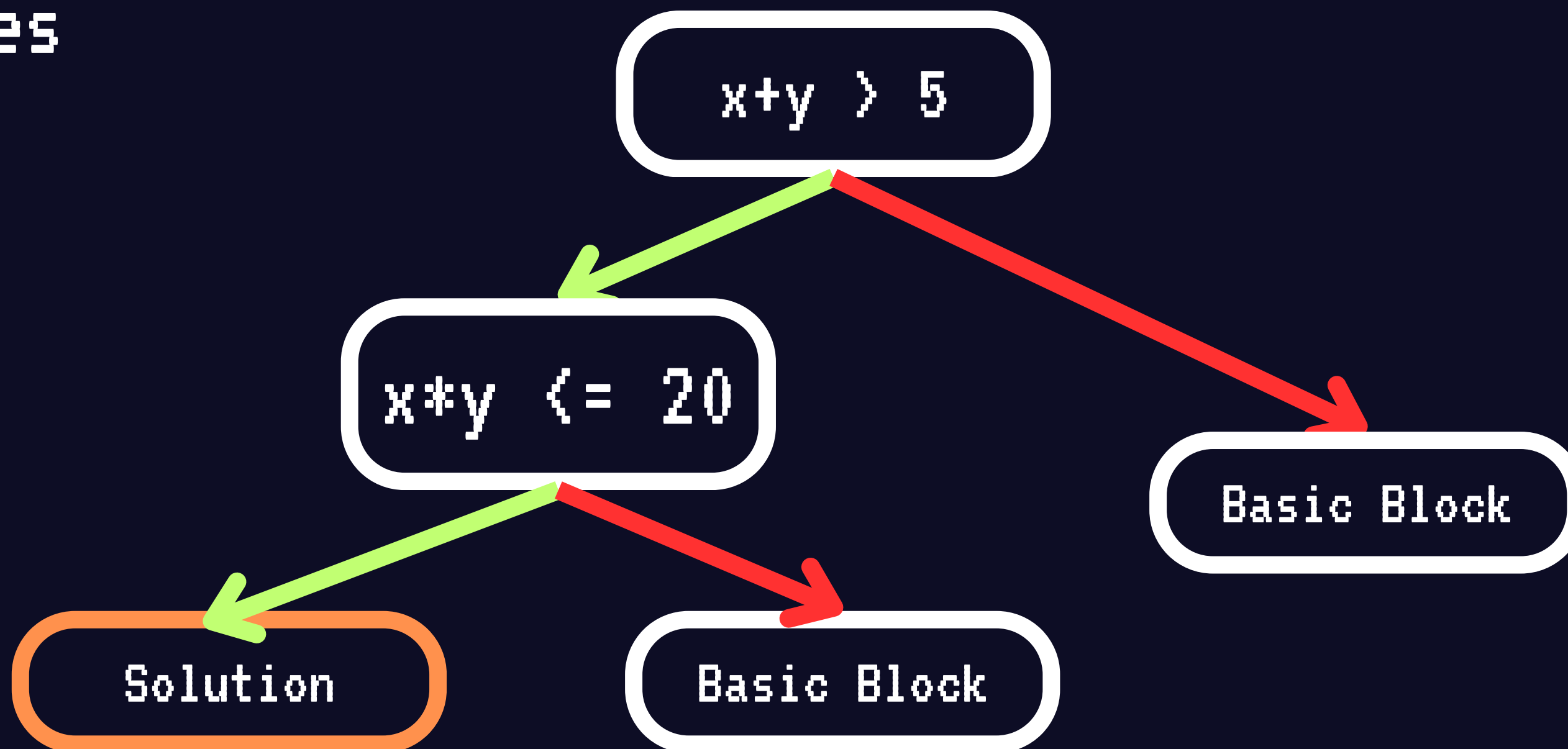
A problem with constraints and symbolic variables
that asks what values satisfy a given path

$(x+y > 5) \text{ AND } (x*y \leq 20)$

Symbolic Execution

SMT Tree:

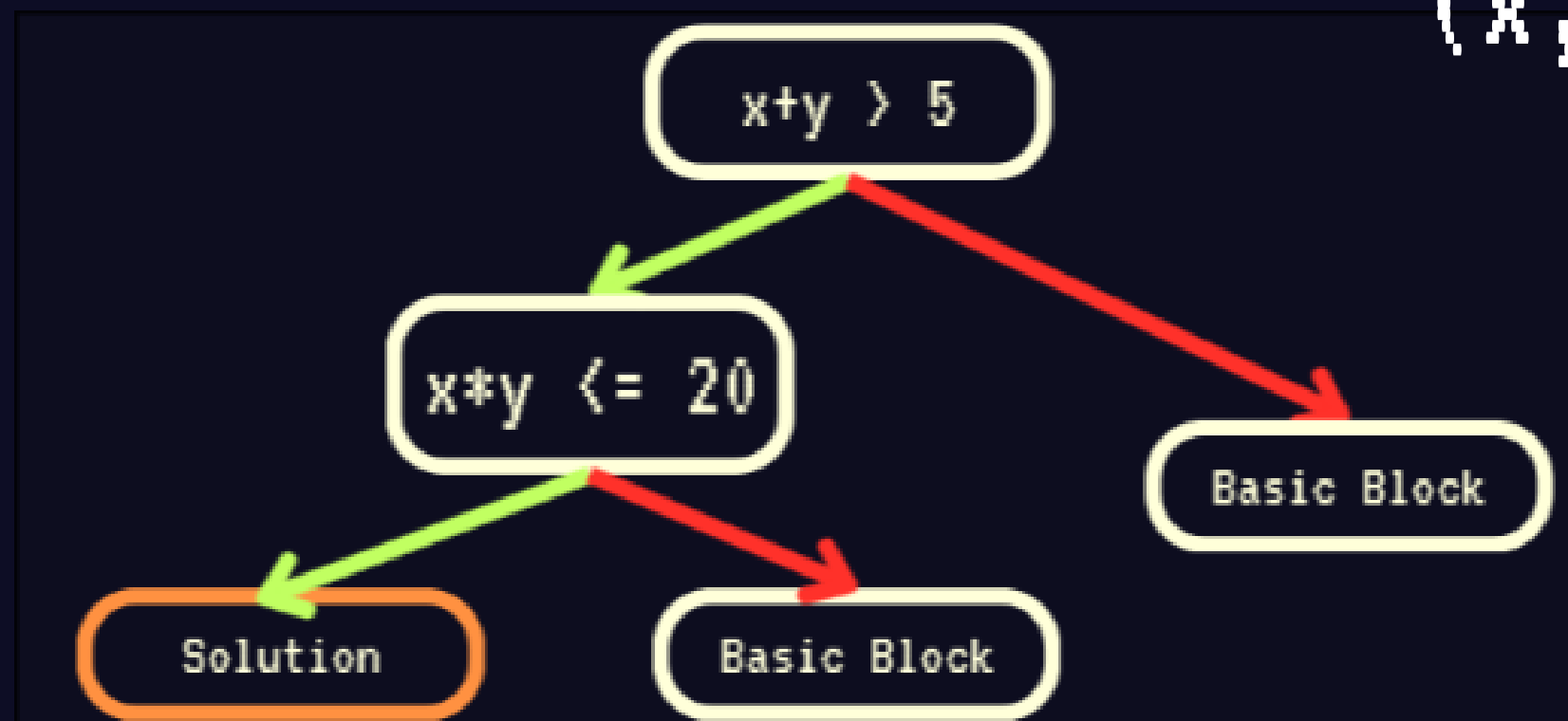
A tree is often generated to represent the possible branches



Symbolic Execution

SMT Solver:

A solver evaluates a tree for the inputs that lead to correct branch.



$(x,y) = (1,5), (1,6), (1,7), \dots$
 $, (18,1), (19,1), (20,1)$

Symbolic Execution

Tools:

z3:

- Simple SMT
- Used in many proof tools



angr:

- Gather constraints from binary
- Has a GUI version



Symbolic Execution

```
#include "stdio.h"
int main(){
    char userInput[5];
    fgets(userInput, 5, stdin);
    if(userInput[0] * 3 == 306)
    if(userInput[1] + userInput[2] == 160)
    if(userInput[1] * 3 == 324)
    if(userInput[3] - userInput[2] == 51)
    if((userInput[2] & userInput[3]) == 0x24)
    if(userInput[2] << 1 == 104)
    if(userInput[0] + userInput[2] == 154)
    if(userInput[1] + userInput[3] == 211)
    if((userInput[0] ^ userInput[1]) == 0x0a)
    | printf("winner!");
    return 0;
}
```

This problem can be solved via symbolic execution

Symbolic Execution

```
from z3 import *
solver = Solver()
userinput = []
for i in range(4):
    userinput.append(BitVec(f'input{i}', 8)) # 8-bit char
solver.add(userinput[0] * 3 == 306)
solver.add(userinput[1] + userinput[2] == 160)
solver.add(userinput[1] * 3 == 324)
solver.add(userinput[3] - userinput[2] == 51)
solver.add(userinput[2] & userinput[3] == 0x24)
solver.add(userinput[2] << 1 == 104)
solver.add(userinput[0] + userinput[2] == 154)
solver.add(userinput[1] + userinput[3] == 211)
solver.add(userinput[0] ^ userinput[1] == 0x0a)
if solver.check() == sat:
    print("".join([chr(solver.model().eval(bv).as_long()) for bv in userinput]))
```

```
└─(21:42:34)─> python solve.py
fl4g%
```

Thats all for now :)

Find more stuff at <https://uoftctf.org/>

Few more things you can look into:

- Concollic execution
- Emulation
- DRM
- Taint Analysis
- Malware
- Game hacking

Resources

CTF Rev:

- <https://pwnable.kr/>
- <https://crackmes.one/>
- <https://crackmy.app/>
- <https://pwn.college/>
- <https://flare-on.com/>
- <https://ctfsearch.hackmap.win/>

Resources

CTF Rev pt 2:

- <https://flagbot.ch/material/>
- <https://how2rev.github.io/>

Resources

x86 Assembly:

- <https://www.cs.virginia.edu/~evans/cs216/guides/x86.html>
- <https://www.skullsecurity.org/wiki/Assembly>

Resources

Obfuscation:

- <https://breakdev.org/x86-shellcode-obfuscation-part-1/>
- <https://scribe.rip/@112piyushsingh/obfuscation-101-how-code-hides-in-plain-sight-and->

Resources

Decompilers:

- <https://decompilation.wiki/fundamentals/>
- <https://www.backerstreet.com/decompiler/introduction.php>